

Creating Controllers

So far, we have been working with routes and views. We have created routes to handle incoming requests and return views. However, as our application grows, we will need to handle more complex logic. This is where controllers come in.

What Is A Controller?

We talked about the role of controllers when we discussed the MVC design pattern, but just to touch on it a bit more, controllers are classes that group related request handling logic. They are responsible for processing incoming requests, interacting with models, and returning views.

Controllers are stored in the `app/Http/Controllers` directory. You will see that there is a file called `Controller.php` in this directory. This file is the base controller class that all other controllers extend. This class does not contain any methods, but it provides a convenient location to add shared methods that all controllers can use. So every controller that we create will go in this folder and it will extend that base controller class.

Let's create our first controller. We can use the `make:controller` Artisan command to create a new controller. Run the following command in your terminal:

```
php artisan make:controller JobController
```

A Note On Naming Conventions

When creating controllers, it is common to use the singular form of the resource name followed by the word `Controller`. For example, if you are creating a controller to handle jobs, you would name it `JobController`. This is a convention that Laravel uses, but you are free to name your controllers however you like. [Here](#) is a list of Laravel naming conventions that you should follow.

This command will create a new file called `JobController.php` in the `app/Http/Controllers` directory. Open this file and you will see that it contains a class definition that extends the base controller class. It does not have any methods yet.

Adding Methods To The Controller

Let's add a method to our controller that will return a view. We will create a method called `index` that will return the `jobs/index.blade.php` view. Here is what the `JobController` class should look like:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
```

```
class JobController extends Controller
{
    // Display a listing of the resource
    public function index()
    {
        return view('jobs.index');
    }
}
```

We are just returning the `jobs.index` view, just like we did in the route.

Using The Controller In A Route

Now that we have created our controller and added a method to it, we can use it in a route. Open the `routes/web.php` file and update the `/jobs` route to use the `JobController`.

First, we need to import the `JobController` at the top of the file:

```
use App\Http\Controllers\JobController;
```

Then, we can update the `/jobs` route to use the `index` method of the `JobController`:

```
Route::get('/jobs', [JobController::class, 'index']);
```

Here, we are using the `JobController` class and the `index` method as the callback for the route. This is a shorthand syntax for defining a controller action. The first parameter is the controller class, and the second parameter is the method that we want to call.

Now when you visit `/jobs` in your browser, you should see the jobs index view. However, if you have any variables, you will get an error.

Passing Data To Views

We can pass data the same way that we did from within the route. We can either pass an array, use the `with` method, or use the `compact` method. If it is more than one variable, I prefer to use the `compact` method. Here is how you can pass data to the view from the controller:

```
public function index()
{
    $title = 'Available Jobs';
    $jobs = [
        'Software Engineer',
        'Web Developer',
        'Data Scientist',
    ];
}
```

```
        return view('jobs/index', compact('title', 'jobs'));
    }
```

Now the view will have access to the `$title` and `$jobs` variables. You can use these variables in the view the same way that you did before.

Home Controller & View

We have a job controller for the `/jobs` page. Let's create a home controller and view for the home page.

Create Home Controller

Use Artisan to create a new controller:

```
php artisan make:controller HomeController
```

Now we have a new controller at `app/Http/Controllers/HomeController.php`. Open the file and add an `index` method to the class that returns the `index` view:

```
public function index()
{
    return view('pages.index');
}
```

This controller has a single method `index` that returns the `home` view in a `pages` folder.

Create Home View

Create a new view file at `resources/views/pages/index.blade.php`. If you still have the `welcome.blade.php` file, you can rename and move that and delete everything in it. Add the following code to the file:

```
<h1>Welcome to Workopia</h1>
<p>Find your dream job today</p>
```

Update Routes

Open the `routes/web.php` file and DELETE the current homepage route and add the following code:

```
use App\Http\Controllers\HomeController;

Route::get('/', [HomeController::class, 'index']);
```

Now when you visit the homepage, you should see the welcome message.